

# VidyaTrack Frontend Architecture

## Document Control

- Product: `VidyaTrack Frontend`
- Audience: Frontend engineers, full-stack engineers, engineering managers, and onboarding junior developers
- Scope: Application architecture, routing, state management, feature boundaries, coding conventions, and extension guidelines
- Source of truth: Current repository structure and implementation in `src/`

## Executive Summary

VidyaTrack frontend is a React + TypeScript single-page application organized around role-based product surfaces: `auth`, `platform`, `management`, `principal`, `teacher`, and `students`. The codebase uses a feature-oriented architecture with three clear layers:

- Route and page orchestration
- Feature-local UI, hooks, helpers, utilities, constants, and types
- Shared platform services such as API clients, query keys, auth store, and core UI building blocks

The architecture goal is predictable change. User-facing behavior is preserved while code is structured so that:

- route files stay thin
- side effects live in hooks
- business rules live in helpers
- formatting and transformation live in utils
- UI stays presentation-first
- shared conventions are enforced through common query keys and scoped API helpers

## Technology Stack

### Core

- React `19`
- TypeScript `5`

- Vite `7`
- React Router DOM `7`
- TanStack React Query `5`
- Zustand `5`
- React Hook Form `7`
- Tailwind CSS `4`
- Axios

## Design and UI

- Tailwind utility styling
- `lucide-react` and `react-icons`
- shared internal UI primitives under `src/components/ui`

## Application Boot Sequence

Application startup begins in `src/main.tsx`.

- `AppProviders` mounts global providers
- dev-only auth API hooks are exposed for console testing
- the app then mounts the router and feature surfaces

Provider composition lives in `src/app/providers`.

Responsibilities:

- query client provider
- router provider
- any application-wide wrappers needed before route rendering

## Top-Level Structure

### **`src/app`**

Contains application assembly.

- `router.tsx`: root browser router composition
- `routes/`: route-tree modules split by role surface
- `providers`: app-wide providers

### **`src/api`**

Contains all API access logic.

- endpoint-specific request functions
- shared `apiClient`
- scoped parameter helpers like `schoolParams`

## **`src/components`**

Contains reusable presentational building blocks.

- `feedback/`: loading, error, empty, and error-boundary components
- `ui/`: primitive reusable controls
- `teachers/`: shared teacher-facing chrome reused across features

## **`src/constants`**

Contains reusable constants and query key factories.

- `queryKeys.ts`: canonical query key definitions
- domain constants like exam types

## **`src/helpers`**

Contains pure shared business logic helpers.

- school context assertions
- exam helpers

## **`src/hooks`**

Contains shared hooks reused across features.

- data fetching
- shared mutations
- context synchronization hooks

## **`src/features`**

Contains role- or domain-specific product features.

Each feature is structured around:

- `components/`
- `hooks/`
- `helpers/`
- `utils/`

- ``constants/``
- ``types/``

### **``src/layouts``**

Contains role layouts and layout internals.

- app shell orchestration
- header/sidebar decomposition
- role-specific layout wrappers

### **``src/store``**

Contains application state stores.

- auth and selected school context

## **Routing Model**

Routing is modularized under ``src/app/routes``.

### **Route Modules**

- ``auth.routes.tsx``
- ``platform.routes.tsx``
- ``teacher.routes.tsx``
- ``principal.routes.tsx``
- ``management.routes.tsx``

### **Why this matters**

- route trees are readable by role
- permissions and layouts are localized
- new role surfaces can be added without bloating one central router file

### **Route Guards**

Role protection is enforced through ``RoleGuard``.

Each protected surface wraps the matching layout:

- super admin -> ``PlatformLayout``
- management -> ``ManagementLayout``

- principal -> `PrincipalLayout`
- teacher -> `TeacherLayout`

## Layout Architecture

Layouts are intentionally split into orchestration and UI pieces.

### `AppShell`

Responsibilities:

- drawer and sidebar state
- breadcrumb derivation
- hard-stop and offline handling
- idle timeout and logout orchestration

Internal structure:

- `layouts/appShell/hooks`
- `layouts/appShell/components`
- `layouts/appShell/helpers`

### Header and Sidebar

Header and sidebar are decomposed into:

- lightweight view components
- isolated formatting utilities
- helper functions for nav state and labels

This keeps navigation behavior stable while allowing targeted UI maintenance.

## Feature Architecture Pattern

The dominant pattern in the repo is:

- thin route shell
- feature hook for orchestration
- pure UI components
- helpers for domain rules
- utils for formatting/transformations
- types for view contracts

- constants for stable configuration

## **Example Flow**

Teacher communications:

- route shell renders success/error gates and panels
- hook manages selected tab, recipient selection, note state, and mutations
- tab panels are split into composer, selector, timeline, and alert sections

## **Why this pattern is used**

- easier testing
- lower merge conflict risk
- easier onboarding for junior developers
- fewer hidden side effects in UI files

## **State Management Strategy**

### **Server State**

React Query is the source of truth for remote data.

Use cases:

- entity lists
- history timelines
- setup readiness
- dashboard metrics
- mutation status and invalidation

### **Client State**

Local component state is used for:

- form input
- wizard progression
- modal visibility
- non-persistent UI filters

### **Global State**

Zustand store is used for:

- authenticated user context
- selected school context
- role and school mappings

## Query Key Strategy

``src/constants/queryKeys.ts`` is the canonical source for shared query keys.

Why this is important:

- avoids inconsistent invalidation
- prevents accidental cache collisions
- makes cross-feature refetch predictable
- provides a reusable contract for future contributors

Current key families include:

- academic setup
- classes, sections, subjects
- students and section-scoped student lists
- teachers
- teaching assignments
- management principal
- attendance
- marks
- communications history
- student notes
- platform school dashboard

## School Context Strategy

VidyaTrack is multi-tenant by school. School context is treated as mandatory wherever data is school-scoped.

Shared enforcement utilities:

- ``requireSchoolId``
- ``schoolParams``

## Rules

- hooks should not duplicate school-context validation if the helper already covers it
- APIs should shape `school\_id` consistently
- invalidation should use school-aware query keys

## **API Layer Design**

API modules are intentionally thin.

Responsibilities:

- endpoint selection
- request parameter shaping
- response typing

Non-responsibilities:

- feature orchestration
- UI copy
- route navigation

## **Current API conventions**

- all network calls flow through `apiClient`
- school-scoped params use `schoolParams`
- endpoint modules are grouped by domain

## **Error Handling Strategy**

### **UI Level**

- feature shells render `LoadingState`, `ErrorState`, or `EmptyState`
- optimistic UI is limited and explicit

### **Hook Level**

- hooks convert API failures into stable UI flags
- where backend returns expected validation conditions like `422`, hooks may normalize to safe empty results

### **Boundary Level**

- route trees use `AppErrorBoundary`



## **Forms and Mutations**

### **React Hook Form**

Used for structured data entry flows:

- onboarding
- management staff creation
- student creation flows

### **Mutation Guidelines**

- construct payloads in hooks, not in page markup
- keep school context explicit
- invalidate only the affected query families
- keep success/reset behavior identical to the original UX unless explicitly changing requirements

## **Communications Subsystem**

Teacher and principal communications follow a shared design:

- homework broadcast
- parent messaging
- student private notes

Shared design principles:

- selection state in hooks
- histories loaded via dedicated hooks
- timeline UI separated from composer UI
- note search and note timeline split for readability

## **Marks and Attendance Subsystems**

Both subsystems use capped concurrency for record creation/upserts through ``runWithConcurrency``.

## **Why**

- avoids server overload
- stabilizes batch-submit behavior
- keeps mutation sequencing explicit

## **Design rule**

Per-record operations and final submit operations remain separate because backend side effects differ.

## **Platform Surface**

Platform is the super-admin surface.

Major areas:

- schools list
- school detail
- school onboarding wizard

The onboarding wizard is split into:

- step header
- step content
- step alerts
- step footer
- reusable selection grid
- review summary cards

## **Management Surface**

Management handles institutional setup.

Major areas:

- dashboard
- principal onboarding
- classes and sections
- teacher onboarding
- subjects
- assignment mapping
- students setup and import

The management setup area now follows consistent feature-local boundaries, making it easier to extend without reopening large route files.

## **Principal Surface**

Principal focuses on oversight and history.

Major areas:

- dashboard
- attendance history
- marks history
- communications
- student and teacher drill-down

The principal communications hook is now split into:

- delivery flow
- notes flow
- shared helper derivations

## **Teacher Surface**

Teacher features include:

- dashboard
- attendance entry
- marks entry
- communications
- student drill-down

The teacher surface now uses shared top-bar chrome from ``src/components/teachers``.

## **Folder Structure Conventions**

**Promote to top-level ``src/*`` only when:**

- the logic is reused across multiple features
- the abstraction is stable
- the naming is domain-neutral enough to remain discoverable

### **Keep feature-local when:**

- the logic is specific to one route or flow
- the abstraction encodes feature-only business rules
- reuse is speculative rather than actual

## **Adding a New Feature**

Recommended flow:

1. create a route shell or feature entry file
2. create a feature-local ``hooks/`` file for orchestration
3. split UI into small components by responsibility
4. move pure rules to ``helpers/``
5. move formatting to ``utils/``
6. extract shared types and constants only when needed
7. use ``queryKeys`` for all shared remote cache identities
8. use ``requireSchoolId`` and ``schoolParams`` where school scoping applies

## **How to Explain This to a Junior Developer**

Use this mental model:

- pages decide what to show
- hooks decide how the page behaves
- helpers decide the rules
- utils clean and format data
- APIs only talk to the server
- query keys tell React Query what each remote cache entry means

Teaching heuristic:

- if code touches rendering and business logic at the same time, split it
- if code has side effects, prefer hooks
- if code is pure and reusable, prefer helpers or utils
- if a value identifies remote data, define its query key centrally

## **Operating Principles for Contributors**

- preserve user behavior first
- prefer extraction over rewrites
- do not hide domain logic in JSX
- avoid introducing shared abstractions until reuse is real
- invalidate the smallest correct query scope
- keep route files readable in under a minute

## **Build and Performance Notes**

Current build uses Vite manual chunking for safe bundle splitting without changing runtime UX.

Chunking intent:

- React core
- router
- state libraries
- UI/vendor utilities
- canvas UI package
- general vendor fallback

This reduces single-bundle concentration while preserving route behavior.

## **Current Repository Health**

Current state after refactor:

- route layer modularized
- feature layer decomposed
- shared hooks standardized
- school-scoped API usage normalized
- lint clean
- build clean

Remaining non-architectural work:

- optional future performance tuning through deeper route-level lazy loading
- optional test coverage expansion where the product team wants stronger regression protection

## Recommended Onboarding Sequence for New Engineers

1. read ``src/app/router.tsx`` and ``src/app/routes/*``
2. read ``src/layouts/*``
3. read ``src/store/auth.store``
4. read ``src/constants/queryKeys.ts``
5. inspect one end-to-end feature per role
6. inspect shared hooks under ``src/hooks``
7. inspect API modules under ``src/api``

## File Reference Index

Key reference points:

- ``src/app/router.tsx``
- ``src/app/routes/auth.routes.tsx``
- ``src/app/routes/platform.routes.tsx``
- ``src/app/routes/teacher.routes.tsx``
- ``src/app/routes/principal.routes.tsx``
- ``src/app/routes/management.routes.tsx``
- ``src/constants/queryKeys.ts``
- ``src/helpers/requireSchoolId.ts``
- ``src/api/helpers/schoolParams.ts``
- ``src/store/auth.store.ts``
- ``src/layouts/AppShell.tsx``
- ``src/features/teachers/dashboard/TeacherDashboard.tsx``
- ``src/features/principal/PrincipalCommunicationsPage.tsx``
- ``src/features/platform/PlatformCreateSchoolPage.tsx``

## Conclusion

The frontend is now structured to support safe scale:

- clear role surfaces
- consistent feature boundaries
- reusable shared data conventions
- reduced route and hook complexity
- maintainable onboarding path for future engineers